

Code Explanation Line by Line

```
import numpy as np
```

Explanation:

Imports NumPy library for:

- Arrays
 - Matrix multiplication
 - Mathematical calculations
-

Define Network Parameters

```
input_neuron = 2
```

Explanation:

- Two input neurons.
- XOR has two inputs:

X1 and X2

```
hidden_neuron = 4
```

Explanation:

- Four hidden neurons used.
 - Hidden layer helps solve non-linear XOR problem.
-

```
output_neuron = 1
```

Explanation:

- One output neuron for XOR output.
-

```
learning_rate = 0.1
```

Explanation:

Controls speed of learning.

```
epochs = 10000
```

Explanation:

Total training iterations.

Training Data

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
```

Explanation:

XOR input combinations.

```
Y = np.array([[0], [1], [1], [0]])
```

Explanation:

Expected XOR outputs.

Initialize Weights

```
W1 = np.random.randn(input_neuron, hidden_neuron) * 0.01
```

Explanation:

Random weights between:

- Input layer
- Hidden layer

Shape:

2×4

```
W2 = np.random.randn(hidden_neuron, output_neuron) * 0.01
```

Explanation:

Random weights between:

- Hidden layer
- Output layer

Shape:

4×1

Initialize Biases

```
b1 = np.random.randn(1, hidden_neuron) * 0.01
```

Explanation:

Biases for hidden layer.

```
b2 = np.random.randn(1, output_neuron) * 0.01
```

Explanation:

Bias for output layer.

Sigmoid Function

```
def sigmoid(x):
```

Explanation:

Defines sigmoid activation function.

```
    return 1 / (1 + np.exp(-x))
```

Explanation:

Sigmoid formula:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid Derivative

```
def sigmoid_derivative(x):
```

Explanation:

Derivative of sigmoid function.

Used in:

- Backpropagation
 - Gradient calculation
-

```
    return x * (1 - x)
```

Explanation:

Derivative formula:

$$\sigma'(x) = x(1 - x)$$

Training Loop

```
for i in range(epochs):
```

Explanation:

Runs training for 10000 iterations.

Forward Pass

```
hidden_layer_input = np.dot(X, W1) + b1
```

Explanation:

Calculates hidden layer input.

Formula:

$$Z = W \cdot X + b$$

```
hidden_layer_output = sigmoid(hidden_layer_input)
```

Explanation:

Applies sigmoid activation to hidden layer.

```
output_layer_input = np.dot(hidden_layer_output, W2) + b2
```

Explanation:

Calculates output layer input.

```
output_layer_output = sigmoid(output_layer_input)
```

Explanation:

Final prediction generated.

Backward Pass

```
output_error = Y - output_layer_output
```

Explanation:

Calculates prediction error.

```
output_delta = output_error * sigmoid_derivative(output_layer_output)
```

Explanation:

Computes output layer gradients.

```
hidden_error = output_delta.dot(W2.T)
```

Explanation:

Sends error backward to hidden layer.

```
hidden_delta = hidden_error * sigmoid_derivative(hidden_layer_output)
```

Explanation:

Computes hidden layer gradients.

Update Weights

```
W2 += np.dot(hidden_layer_output.T, output_delta) * learning_rate
```

Explanation:

Updates weights between:

- Hidden layer
 - Output layer
-

```
b2 += np.sum(output_delta, axis=0, keepdims=True) * learning_rate
```

Explanation:

Updates output bias.

```
W1 += np.dot(X.T, hidden_delta) * learning_rate
```

Explanation:

Updates weights between:

- Input layer
 - Hidden layer
-

```
b1 += np.sum(hidden_delta, axis=0, keepdims=True) * learning_rate
```

Explanation:

Updates hidden bias.

Testing Phase

```
x_test = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
```

Explanation:

Testing XOR inputs.

```
y_test = np.array([[0], [1], [1], [0]])
```

Explanation:

Expected outputs.

Final Prediction

```
output_layer_output = sigmoid(output_layer_input)
```

Explanation:

Network predicts final outputs.

Print Results

```
print("Input:")
```

Displays input values.

```
print("Output:")
```

Displays predicted outputs.

```
print("Expected Output:")
```

Displays actual XOR outputs.

What Happens Internally in This Code? (Detailed Explanation)

This program implements a Backpropagation Neural Network to solve the XOR problem. First, NumPy library is imported for matrix operations and mathematical calculations. The XOR dataset is then created with four input combinations and corresponding outputs. Since XOR is a non-linear problem, a hidden layer with 4 neurons is added to help the network learn complex patterns.

The neural network initializes random weights and biases between layers. During Forward Propagation, the inputs move from input layer to hidden layer and finally to output layer. At each layer, weighted sum and bias are calculated, and sigmoid activation function is applied. This generates predicted outputs between 0 and 1.

After prediction, Backpropagation starts. The network compares predicted output with actual output and calculates error. This error is sent backward through the network. Using sigmoid derivative, gradients are calculated for both hidden and output layers. These gradients help determine how much the weights and biases should change.

The weights and biases are updated repeatedly for 10000 epochs using learning rate. With continuous training, the prediction error decreases gradually and the model learns correct XOR outputs. Finally, the trained network is tested using all XOR input combinations, and the predicted outputs become very close to the expected outputs.

Viva / Oral Questions and Answers

1. What is XOR function?

XOR (Exclusive OR) is a logic gate that gives output 1 when both inputs are different and 0 when both inputs are same.

2. What are the XOR truth table outputs?

X1	X2	Y
----	----	---

0	0	0
---	---	---

X1 X2 Y

0 1 1

1 0 1

1 1 0

3. Why is XOR problem important in ANN?

Because XOR is a non-linear problem and cannot be solved using a single perceptron.

4. Why cannot a single perceptron solve XOR?

Because XOR data is not linearly separable.

5. What is linearly separable data?

Data that can be separated using a single straight line is called linearly separable data.

6. What is Backpropagation?

Backpropagation is a learning algorithm used to reduce prediction error by updating weights and biases.

7. What is Forward Propagation?

Forward propagation is the process where inputs move from input layer to output layer to generate prediction.

8. What is the role of hidden layer in XOR problem?

Hidden layer helps neural network learn non-linear patterns.

9. How many hidden neurons are used in this code?

4 hidden neurons

10. Which activation function is used in this practical?

Sigmoid activation function.

11. Write sigmoid activation function formula.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

12. What is the range of sigmoid function?

0 to 1

13. Why is sigmoid function used?

Because it converts outputs into probabilities and works well for binary classification.

14. What is derivative of sigmoid function?

$$\sigma'(x) = x(1 - x)$$

15. Why is sigmoid derivative needed?

It is needed during backpropagation for gradient calculation.

16. What is learning rate?

Learning rate controls how much weights are updated during training.

17. What is epoch?

One complete training cycle of the dataset is called an epoch.

18. How many epochs are used in this program?

10000 epochs

19. What is bias in neural network?

Bias helps shift activation function and improves learning capability.

20. Why are weights initialized randomly?

Random initialization helps the network learn properly and prevents same neuron behavior.

21. What is loss or error?

Loss is the difference between actual output and predicted output.

22. What is the purpose of backpropagation?

To minimize error and improve prediction accuracy.

23. What does np.dot() do?

It performs matrix multiplication.

24. What is the use of NumPy in this program?

NumPy is used for:

- Arrays
 - Matrix operations
 - Mathematical calculations
-

25. What is neural network?

A neural network is a computational model inspired by the human brain used for machine learning tasks.

26. What type of learning is used in this practical?

Supervised learning.

27. What are input neurons in this code?

2 input neurons

Represent:

X1 and X2

28. What is output neuron?

Neuron that gives final prediction.

29. What is the purpose of training the network?

To learn correct XOR outputs by reducing error.

30. What is the main objective of this practical?

To implement XOR function using Backpropagation Neural Network in Python.